Лекція №1 (2 години). Архітектура WWW та протоколи передачі даних

План лекції

1. Глобальна інфраструктура
2. Клієнт-серверна парадигма та модель взаємодії
3. Анатомія HTTP-повідомлень (Методи та Статус-коди)
4. Семантичний Веб (Web 3.0) та Децентралізація (Web3)

Перелік умовних скорочень

- **WWW (World Wide Web)** — Всесвітня павутина, інформаційна система, пов'язана гіпертекстовими посиланнями.
- **HTML (HyperText Markup Language)** — мова гіпертекстової розмітки.
- **HTTP (HyperText Transfer Protocol)** — протокол передачі гіпертексту.
- **HTTPS (HyperText Transfer Protocol Secure)** — розширення протоколу HTTP, що підтримує шифрування (TLS/SSL).
- **DNS (Domain Name System)** — система доменних імен, що перетворює імена хостів у IP-адреси.
- **IP (Internet Protocol)** — мережевий протокол, що використовується для маршрутизації даних.
- **TCP (Transmission Control Protocol)** — протокол управління передачею даних є надійним транспортним механізмом.
- **URI / URL (Uniform Resource Identifier / Locator)** — уніфікований ідентифікатор / локатор ресурсу.
- **API (Application Programming Interface)** — програмний інтерфейс додатку.
- **TLS (Transport Layer Security)** — криптографічний протокол, що забезпечує безпеку зв'язку.

Вступ

Веб-розробка — це складна інженерна дисципліна, що виходить далеко за межі простого знання мов програмування чи інструментів розмітки. Це створення інформаційних взаємодій, в основі яких лежить надійна, гнучка та стандартизована архітектура: архітектура Всесвітньої павутини (World Wide Web або WWW) та її фундамент — Інтернет.

Інтернет — це глобальна комп'ютерна мережа, яка з'єднує мільйони локальних і регіональних мереж ("апаратна" складова), тоді як Всесвітня павутина (WWW) — це система гіпертекстових документів, що знаходяться в цій мережі ("програмна" або "інформаційна" складова). Для того, щоб веб-розробник міг створювати дійсно швидкі, безпечні та надійні додатки, йому життєво необхідно розуміти, як ці дві складові взаємодіють. Розуміння фундаментальних принципів передачі даних в Інтернеті — це база для оптимізації швидкості завантаження, побудови надійної архітектури API, безпечної передачі конфіденційних даних та вирішення найрізноманітніших технічних проблем, що виникають в процесі розробки.

1. Глобальна інфраструктура

Інтернет працює як сукупність мереж, які взаємодіють за єдиними правилами. Цю архітектуру часто описують багаторівневими моделями, найвідомішими з яких є модель OSI (7 рівнів) та стек протоколів TCP/IP (4 рівні). Сучасний інтернет у переважній більшості базується на моделі TCP/IP, яка є більш практичною:

1. **Рівень мережевого доступу (Network Access/Link Layer):** Відповідає за фізичну передачу даних (Ethernet, Wi-Fi, MAC-адреси).
2. **Міжмережевий рівень (Internet Layer):** Ключовий рівень, який відповідає за логічну адресацію та маршрутизацію. Головний протокол — **IP (Internet Protocol)** (IPv4 або IPv6). Він доставляє пакети даних від комп'ютера А до комп'ютера Б, де б вони не знаходились.
3. **Транспортний рівень (Transport Layer):** Відповідає за передачу даних між конкретними програмами (процесами).
   - **TCP (Transmission Control Protocol):** Гарантує доставку без втрат і в правильному порядку. Використовується для WWW (HTTP), електронної пошти (SMTP). Він створює сесію, розбиває дані на пакети, стежить, щоб вони всі дійшли і "зшиває" їх назад.
   - **UDP (User Datagram Protocol):** Не гарантує доставку (пакети можуть губитися), зате працює дуже швидко. Ідеальний для стрімінгу відео, голосового зв'язку чи онлайн-ігор.
4. **Прикладний рівень (Application Layer):** Тут працюють програми та їх логіка. Тут знаходяться протоколи:
   - **HTTP/HTTPS** — для веб-сторінок.
   - **DNS** — для розпізнавання імен.
   - **FTP** — передача файлів.

Порти: Коли комп'ютер отримує TCP пакет, як він розуміє, якій програмі його віддати (браузеру чи поштовому клієнту)? Для цього існують порти — числові ідентифікатори процесів (від 0 до 65535).

- **80** — Звичайний HTTP (незахищений веб-трафік).
- **443** — Захищений HTTPS (увесь сучасний веб-трафік).
- **21 / 22** — FTP / SSH (передача файлів та віддалене управління сервером).
- **3306 / 5432** — Бази даних (MySQL / PostgreSQL).
- **3000 / 5173 / 8080** — Часто використовуються розробниками для локальних серверів (React, Node.js).

> ***Практика:*** *Ви можете побачити всі відкриті порти та програми, які їх використовують, на своєму комп'ютері. Відкрийте командний рядок (Terminal/CMD) і введіть команду* `netstat -ano` *(Windows) або* `lsof -i -P -n` *(Mac/Linux). Ви побачите список IP-адрес і портів, які зараз "слухає" ваша система.* ***Практика:*** *Ви можете побачити всі відкриті порти та програми, які їх використовують, на своєму комп'ютері. Відкрийте командний рядок (Terminal/CMD) і введіть команду* `netstat -ano` *(Windows) або* `lsof -i -P -n` *(Mac/Linux). Ви побачите список IP-адрес і портів, які зараз "слухає" ваша система.*

2. Клієнт-серверна парадигма та модель взаємодії

Архітектура веб-додатків більшою частиною спирається на клієнт-серверну модель. Це розподілена архітектура, в якій є **клієнт**, що робить запит на послуги (або ресурси), і **сервер**, який надає (обслуговує) ці запити.

2.1 Розподіл обов'язків у клієнт-серверній архітектурі

- **Клієнт (Client):** Ініціатор запиту. В розрізі веб-розробки, клієнтом найчастіше виступає веб-браузер (Chrome, Firefox, Safari) або мобільний додаток, або скрипт. Клієнт форматує запит (відповідно до протоколу HTTP), відправляє його на сервер, отримує відповідь від сервера та рендерить (відмальовує) отриманий HTML/CSS/JS код користувачеві в зручному графічному інтерфейсі. Клієнт не тримає даних довгостроково.
- **Сервер (Server):** Програма (або апаратний комп'ютер), яка постійно працює ("прослуховує" певний порт) та чекає на вхідні запити. Коли сервер отримує HTTP запит, він аналізує його, виконує необхідну логіку (наприклад, звертається до бази даних PostgreSQL за інформацією про товари), генерує відповідь (HTTP-відповідь) і відправляє її назад клієнту.

Сучасний веб еволюціонує: якщо раніше сервери генерували готові HTML сторінки (Server-Side Rendering — SSR, на базі PHP/Java), то зараз все частіше сервер виступає лише в ролі "API" (віддає сухі дані у форматі JSON), а клієнтський браузер за допомогою JavaScript (React, Vue, Angular) самостійно "будує" HTML на їх основі (Client-Side Rendering — CSR). Це називається архітектура Single Page Application (SPA).

2.2 Система доменних імен (DNS)

Коли ви вводите `https://blog.google.com`, ваш комп'ютер не знає, де знаходиться цей сервер. Протокол IP працює тільки з числовими IP-адресами (наприклад `142.250.217.206`). **DNS (Domain Name System)** — це як "телефонна книга" інтернету.

Доменне ім'я читається зліва направо (від найвищого рівня до найнижчого) і має чітку ієрархію:

1. `.` **(Root Domain):** Нульовий рівень. Усі домени неявно закінчуються крапкою.

2. `.com`, `.org`, `.ua` **(TLD - Top Level Domain):** Домен першого рівня. Вказує на географічну або організаційну приналежність.
3. `google`, `wikipedia` **(SLD - Second Level Domain):** Домен другого рівня. Власне унікальне ім'я компанії чи організації.
4. `blog`, `mail`, `www` **(Subdomain):** Піддомен (домен третього рівня). Використовується для логічного поділу великого сайту на частини.

Процес (DNS Resolution - як ми знаходимо IP):

1. Браузер спочатку перевіряє свій власний кеш і кеш ОС (файл `hosts`, де можна локально "підмінити" IP-адресу).
2. Якщо не знаходить, ОС запитує DNS-сервіс інтернет-провайдера (ISP) (або ті, що вказані в налаштуванні роутера, як-от `8.8.8.8` Google).
3. Якщо і там немає — сервер провайдера починає послідовне опитування:
   - **Root DNS Server:** "Де знаходяться сервери, що відповідають за зону `.com`?"
   - **TLD (Top-Level Domain) Server:** Отримавши IP серверів зони `.com`, питає: "Хто відповідає за домен `google.com`?"
   - **Authoritative Name Server:** Отримавши адресу авторитетного сервера Google, питає "Дай мені точний IP для субдомену `blog.google.com`".
4. Точна IP-адреса повертається браузеру і починається встановлення TCP з'єднання (TCP Handshake).

Цей шлях зазвичай займає від 10 до 100 мілісекунд, але саме ця система дозволяє нам запам'ятовувати зручні імена (домени), замість довгих беззмістовних чисел.

2.3 Еволюція протоколів HTTP від 1.1 до 3

Основою WWW є протокол HTTP (Hypertext Transfer Protocol). З часом він розвивався, вирішуючи проблеми продуктивності "важкого" вебу.

- **HTTP/1.1 (1997):** Надійний і простий текстовий протокол. Його головна проблема — Head-of-Line Blocking (Блокування початку черги). Для кожного файлу (HTML, CSS, картинки) потрібно було встановлювати нове (хоч і тимчасово перекористовуване) TCP з'єднання. Оскільки браузери мали ліміт (зазвичай 6 підключень на домен одночасно), завантаження сайту зі 100 картинками перетворювалось на "чергу".
- **HTTP/2 (2015):** Справжній прорив. Ввів концепцію *Мультиплексування* — тепер всі запити ідуть через **одне** єдине TCP-з'єднання паралельно, без очікування в черзі. А також стискає всі службові заголовки (Header compression).
- **HTTP/3 (2022+):** Відмовився від TCP! Заснований на новому протоколі **QUIC** (який працює поверх пакета UDP). Відсутність TCP означає ще швидше встановлення з'єднання, адже більше не потрібно витрачати час на "потрійне рукостискання" (3-way handshake) TCP, і навіть відсутність втрати сесії, якщо ви переключилися з Wi-Fi на 4G (UDP пакети знайдуть шлях). Створений для мобільного та надшвидкого вебу.

3. Анатомія HTTP-повідомлень (Методи та Статус-коди)

HTTP є "stateless" протоколом — протоколом без збереження стану. Це означає, що кожен запит від клієнта до сервера не пов'язаний з попереднім. Сервер "не пам'ятає" вас (це робиться штучно через Cookies та сесії). Дані передаються у формі "Повідомлень" ("Запит" від клієнта та "Відповідь" від сервера).

3.1 Методи запиту та їх семантика

Кожен HTTP-запит має *метод* — дієслово в наборі команд, що позначає намір клієнта. В архітектурі REST API ці методи зіставляються з діями CRUD (Create, Read, Update, Delete):

- **GET** — запит на Читання (Read) інформації. Використовується для отримання HTML-сторінок, зображень або JSON даних. **Ніколи не повинен змінювати стан сервера** (безпечний метод).
- **POST** — запит на Створення (Create) нового ресурсу (напр. створення нового замовлення, реєстрація). Відповідні дані передаються в тілі запиту (Request Body), і вони можуть бути захищені TLS-шифруванням.
- **PUT** — запит на повне Оновлення (Update/Replace) існуючого ресурсу. Він ідемпотентний (повторний виклик одного PUT матиме той самий ефект, що і один).
- **PATCH** — запит на часткове Оновлення (Update) (напр. зміна тільки пароля або тільки номера телефону юзера).
- **DELETE** — запит на Видалення (Delete) ресурсу за його ідентифікатором (наприклад `/users/12`).
- **OPTIONS** — службовий метод для запиту підтримуваних сервером методів (використовується в механізмі CORS безпеки). Під капотом браузер викликає цей метод кожного разу, коли робить запит POST/PUT на інший домен до чужого сервера, і цей метод повертає статус "OK".

3.2 Класифікація статус-кодів відповідей

Коли сервер формує відповідь, обов'язковим її елементом є тризначний числовий код статусу (Status Code). Він дозволяє браузеру дуже швидко зрозуміти результат операції. Розуміння цих кодів для веб-розробника при дебагінгу є обов'язковим.

- **1xx (Інформаційні):** Сервер отримав запит, процес триває (наприклад 101 Switching Protocols для переходу на WebSockets).
- **2xx (Успішні):**
  - `200 OK` — Класичний успіх. Дані повернуто.
  - `201 Created` — Переважно у відповідь на POST/PUT. Новий ресурс був успішно збережений у базі.
  - `204 No Content` — Успіх, але тіло відповіді порожнє (часто у відповідь на DELETE).
- **3xx (Перенаправлення / Редіректи):**
  - `301 Moved Permanently` — Ресурс назавжди переїхав на новий URL. Сервер поверне заголовок `Location`, і браузер запам'ятає це (кешує). Обов'язкове для SEO

при редизайні сайту.
    - `302 Found` (або `307`) — Тимчасове перенаправлення.
    - `304 Not Modified` — Використовується для кешування. Браузер питає "Картинка `/logo.png` змінювалася з вівторка?", сервер відповідає `304` (Ні, вона точно така сама), і браузер бере власну копію з кешу (економія трафіку).
- **4xx (Помилки клієнта - Client Error):**
    - `400 Bad Request` — Помилка відправлених даних. Клієнт надіслав "кривий" синтаксис (відсутні обов'язкові параметри чи помилковий JSON).
    - `401 Unauthorized` — Необхідно залогінитися (користувач є неавторизованим).
    - `403 Forbidden` — Користувач залогінений, але у нього немає *прав* на цю дію (наприклад, звичайний користувач лізе до панелі Адміна).
    - `404 Not Found` — Відомий усім. Ресурс не існує.
    - `429 Too Many Requests` — Спам. Спрацював ліміт запитів (Rate Limiting).
- **5xx (Помилки сервера - Server Error):**
    - `500 Internal Server Error` — Відбувається, якщо бекенд код впав з помилкою (наприклад помилка у формулі у Node.js).
    - `502 Bad Gateway` / `504 Gateway Timeout` — Часто трапляється, якщо балансувальник навантаження або Nginx працюють, але основний бекенд-сервер відключений чи "завис", і відповіді чекали надто довго.

> ***Як побачити статус-коди самостійно:*** *У будь-якому сучасному браузері відкрийте* ***Developer Tools*** *(Інструменти розробника, клавіша `F12` або `Ctrl+Shift+I`), перейдіть на вкладку* ***Network (Мережа)*** *та оновіть сторінку. Ви побачите водоспад з десятків запитів (документ HTML, CSS файли, картинки, скрипти). У колонці "Status" буде чітко видно числові коди — 200 (зелені), 304 (взяті з кешу), або червоні (404, якщо картинку не знайдено). Клікнувши на будь-який запит, у розділі "Headers" ви побачите повну анатомію: URL, метод, заголовки Відповіді та Запиту.*

3.3 Життєвий цикл HTTP-запиту

Давайте складемо все докупи, зімітувавши введення `https://www.example.com` у ваш браузер:

1. Браузер "парсить" URL і виокремлює протокол (`https`), домен (`www.example.com`).
2. Браузер використовує **DNS**, щоб дізнатися IP-адресу сервера. (Наприклад, `192.0.2.1`).
3. Оскільки використовується **HTTPS**, браузер ініціює TCP з'єднання на порт `443` серверу `192.0.2.1` (TCP 3-way Handshake).
4. Здійснюється **TLS Handshake**: Клієнт і Сервер узгоджують ключі шифрування, щоб ніхто не міг перехопити дані по дорозі (захист від Man-in-the-Middle атак).
5. Браузер формує та шифрує **HTTP Request** (Метод `GET / HTTP/1.1`) та надсилає інформацію про себе (`User-Agent`, підтримувані формати).

6. Сервер отримує зашифрований пакет, розшифровує, виконує свою логіку (знаходить в папці чи генерує файл `index.html`).
7. Сервер формує та шифрує **HTTP Response** (`HTTP/1.1 200 OK` зі вбудованою сторінкою HTML).
8. Браузер отримує відповідь, закриває з'єднання (або тримає його відкритим за стандартом HTTP/1.1+ (Keep-Alive)) та починає читати HTML код (рендерити сторінку, малювати CSS).

4. Семантичний Веб (Web 3.0) та Децентралізація (Web3)

Історія розвитку "Вебу" часто ділиться на кілька ключових етапів. З урахуванням того, як швидко змінюються технології, важливо розуміти фундаментальні концепції, що стоять за новими етапами розвитку Інтернету.

4.1 Semantic Web

Концепція Web 3.0, (іноді називається Семантичною Павутиною — Semantic Web) фокусується на *значенні* або семантиці інформації. Її головна ідея: Комп'ютери повинні не просто передавати та малювати інформацію (текст і картинки), але і **розуміти її зміст** так само глибоко, як і люди.

- В Web 1.0/2.0 слово "Стів Джобс" — це просто набір літер на сторінці `<h1>Стів Джобс</h1>`.
- У Semantic Web сторінка використовує спеціальні стандарти (наприклад структуровані мікродані Schema.org або RDFa), де чітко сказано: `type: "Person", name: "Steve Jobs", birthDate: "1955-02-24"`.

Це дозволяє пошуковим системам, голосовим асистентам (Siri/Alexa) та штучному інтелекту зв'язувати розрізнені дані по всьому Інтернету в "Граф знань" (Knowledge Graph) і давати точні та персоналізовані відповіді на складні запитання. Еволюція AI, яка відбувається зараз, є апогеєм реалізації бачення Семантичного Вебу.

4.2 Web3 децентралізація та Блокчейн

Термін "Web3" (написання без пробілу та нулів) зазвичай використовується у контексті Децентралізованого Інтернету, заснованого на технологіях блокчейну. Основа сучасного Інтернету Web 2.0 (Google, Meta, Amazon) має недолік — централізація даних на серверах корпорацій. Web3 пропонує зміну парадигми:

- Замість зберігання бази даних на одному сервері Amazon, дані розподілені (блокчейн) між тисячами комп'ютерів.
- Авторизація часто виконується не через Email та Пароль, збережені на чужому сервері, а через ваш особистий криптогаманець (напр. MetaMask, де зберігається ваш приватний ключ).

- Логіка серверного застосунку виконується у вигляді Smart Contracts — відкритих фрагментів коду, завантажених у мережу (наприклад, Ethereum), запуск яких підтверджує вся мережа.

Незважаючи на високу спеціалізацію цих технологій, базові протоколи HTTP/HTTPS залишаються транспортом для комунікації додатків як у Semantic Web, так і в епосі блокчейну.

Висновки

Інтернет та Всесвітня павутина працюють завдяки чіткій та багатошаровій системі протоколів: від фізичної передачі IP-адрес до високорівневого розуміння текстових повідомлень HTTP-сервером. Фундамент сучасного клієнт-серверного зв'язку базується на методах (зрозумілих дієсловах типу GET/POST) та статус-кодах відповіді. Знання цього "життєвого циклу" — це найнадійніша інвестиція в компетентність розробника, адже ці стандарти не змінюються щодня, на відміну від JavaScript фреймворків чи CSS препроцесорів.

Джерела

1. MDN Web Docs: An overview of HTTP (https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview)
2. High Performance Browser Networking by Ilya Grigorik (Чудова книга від експерта Google про архітектуру мереж) (https://hpbn.co/).
3. Офіційна документація MDN: HTTP status codes (https://developer.mozilla.org/en-US/docs/Web/HTTP/Status)
4. Cloudflare Learning Hub: Що таке DNS? (https://www.cloudflare.com/learning/dns/what-is-dns/)
5. Як працює Web: Mozilla Developer Network (https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works)

Запитання для самоперевірки

1. В чому різниця між поняттями Інтернет (Internet) та Всесвітня Павутина (WWW)?
2. Чому протокол TCP/IP називають "надійним", а UDP — "швидким"? Де саме використовуються ці два підходи?
3. Опишіть процес розпізнавання доменного імені за допомогою DNS (від кешу браузера до Root-серверів).
4. Яка проблема протоколу HTTP/1.1 була вирішена за рахунок впровадження мультиплексування в HTTP/2? Які переваги дає HTTP/3 порівняно з TCP-протоколами?
5. Поясніть відмінність між HTTP-методами `POST` та `PUT` в контексті дій.
6. Що означають групи статус-кодів 2xx, 3xx, 4xx і 5xx? Яка ключова різниця між 401 та 403?
7. Чим ідея "Semantic Web (Web 3.0)" (базується на розумінні змісту через мікродані та AI) відрізняється від парадигми "децентралізованого Web3" (сучасний контекст блокчейну)?